

Lecture 8: Core Machine Learning, part 2

LSE ME314: Introduction to Data Science and Machine Learning (<https://github.com/me314-lse>)

2026-07-23

Ryan Hübert

Performance on new data

Underfitting is a problem. . .

So far, we've been very concerned about improving performance:

- Lower MSE on estimated regression models.
- High accuracy, precision and/or recall on classification models.

We're worried about our estimated model **underfitting** the data.

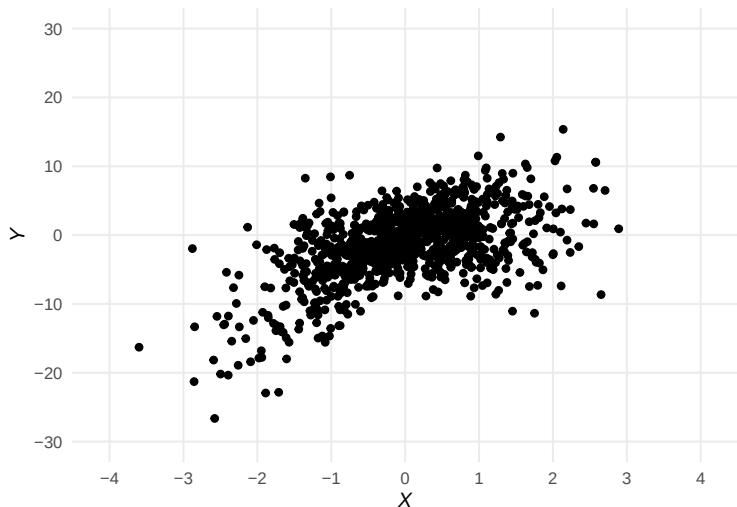
- An underfit model is too simple to capture the real patterns in the data.
- Symptom: predictions far from the actual data (e.g., high MSE), even in the data the machine learned from.

Why are we concerned?

- Because we'd ideally like to use models trained by the machine on labeled data to predict labels in unlabeled data.
- In other words: we want to use the model on other data sets.

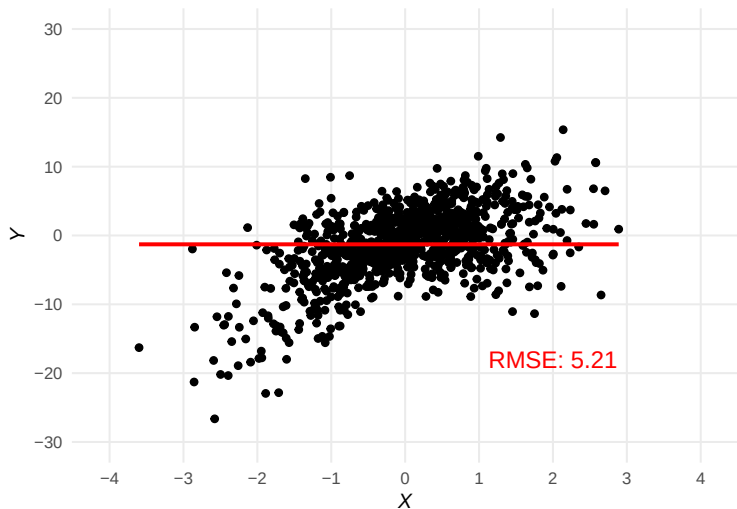
Underfitting is a problem. . .

Remember our labeled data from last lecture



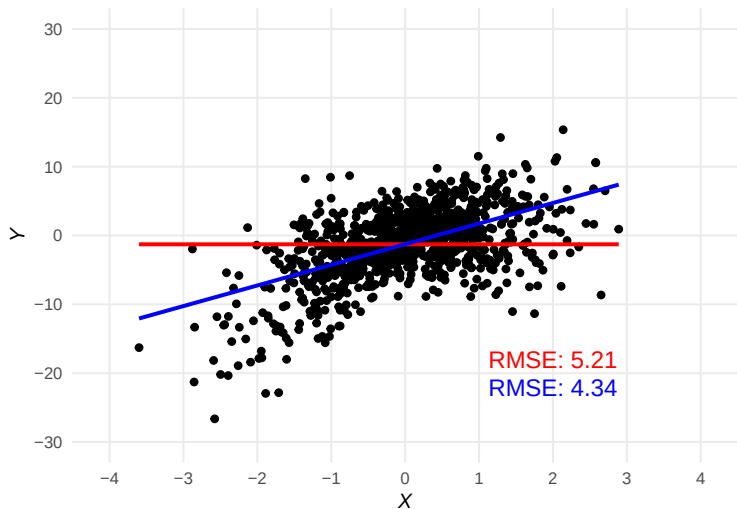
Underfitting is a problem. . .

An egregious example of underfitting: predict with mean of Y .



Underfitting is a problem. . .

An egregious example of underfitting: predict with mean of Y .



... and so is overfitting

Once we acknowledge we want to use our estimated model to do predictions on other datasets, there's another subtler problem.

Machines are very good at finding patterns in data, if allowed to.

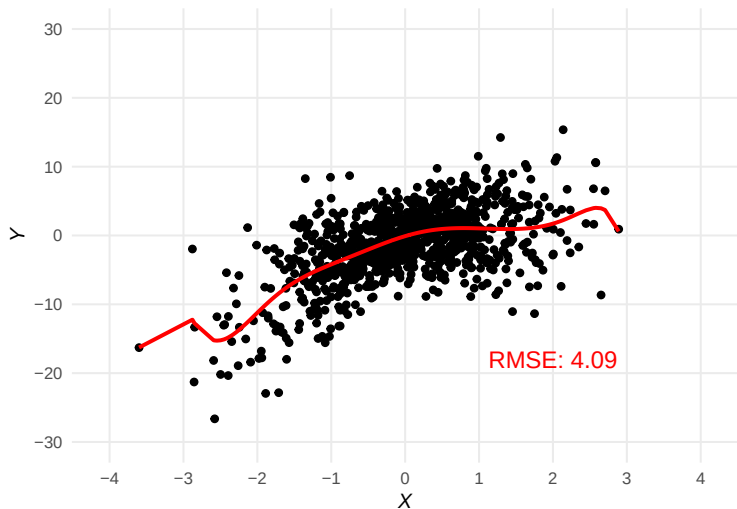
As a result, they tend to **overfit** the data they're given.

What do I mean by this?

- The machine builds a model that is too closely tailored for the sample it used to learn (the “training set”).
- Overfitting means an estimated model won't perform well on new data.

... and so is overfitting

Overfitting example: using a 10th degree polynomial.



Overfitting

It's hard to see why this is a problem: our RMSE is lower!

But, it should be intuitive that this model won't do well on a different dataset.

The RMSE we calculated is measuring **in-sample performance**.

→ Specifically: in the *training* sample the machine used to learn.

Recall: we care about using the estimated model to fill in missing data: meaning predicting Y in *other* datasets

In-sample RMSE does not tell us how the model will do on other data.

Test sets

How can we see whether an estimated model performs well on other datasets?

We test it on other datasets!

A **test set** is a subset of our labeled data that we “hold out.”

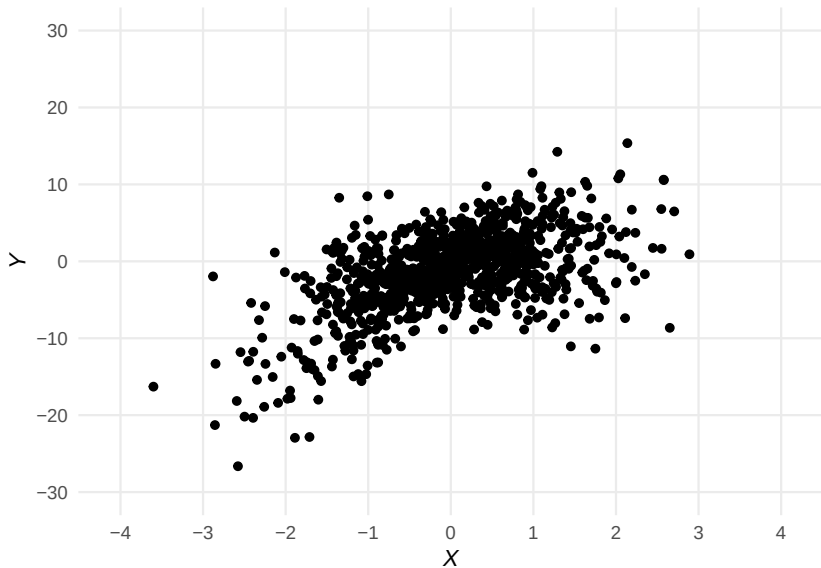
→ The machine is not allowed to look at it, or use it in any way.

The machine builds its model using training data only.

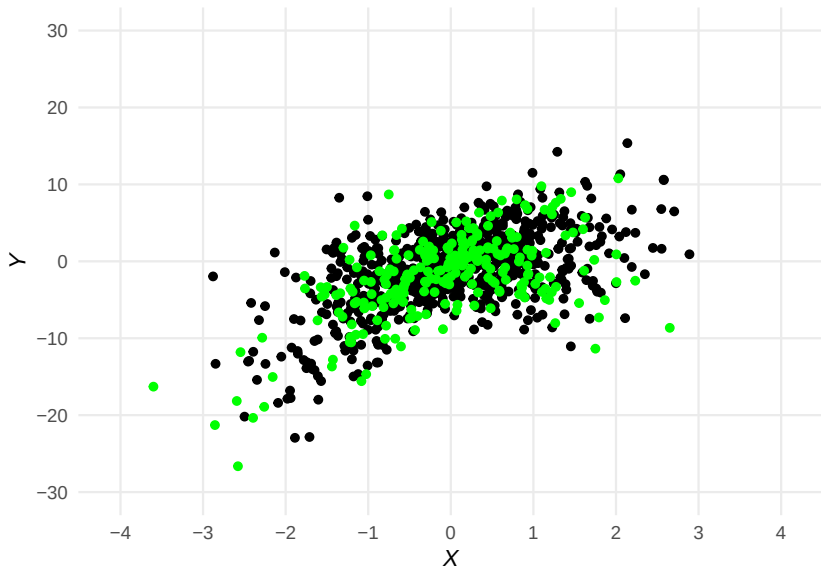
→ It probably will overfit to the training data (at least a little).

→ But we can see how much by looking at how well model does on test data.

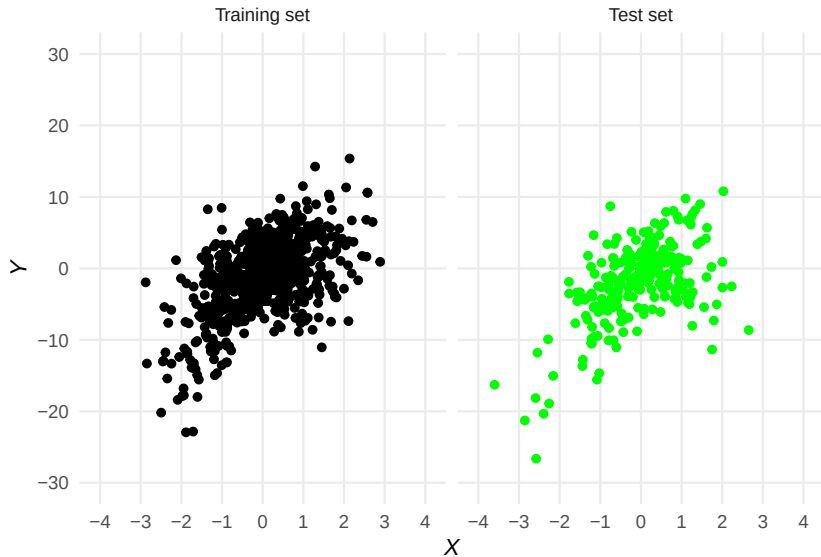
Test sets



Test sets



Test sets



Test sets

Let's calculate RMSE on the training and test sets.

```
# A tibble: 4 x 3
  Model      TrainingRMSE TestRMSE
  <chr>          <dbl>    <dbl>
1 mean           5.09      5.57
2 linear          4.23      4.64
3 quadratic       4.10      4.26
4 poly10          4.03     23.7
```

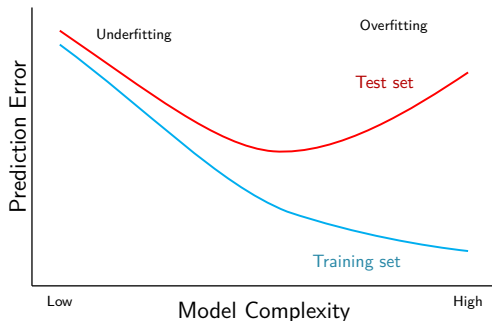
Two observations:

1. As is apparent here: performance is usually *worse* in the test set than in the training set.
2. Even more interestingly, notice: RMSE monotonically decreases in the training set as model becomes more complex, but in the test set, RMSE goes down then back up.

Point 2 is key evidence that 10th degree polynomial is overfitting.

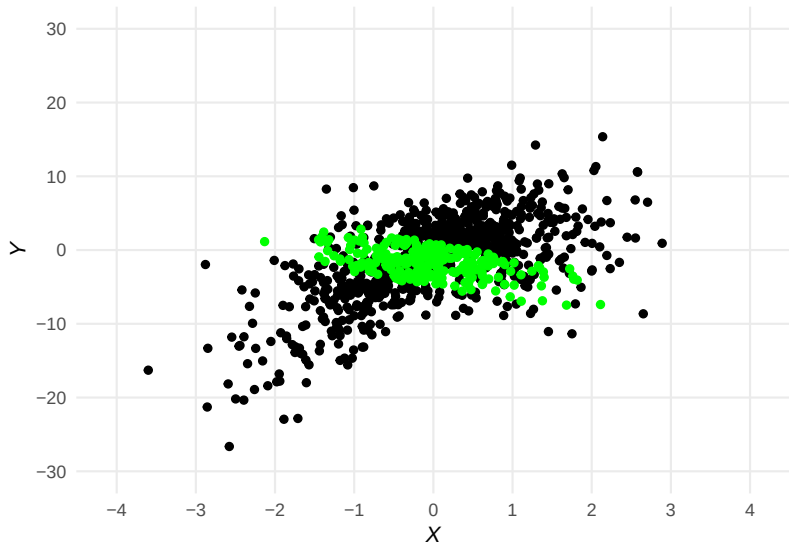
This generalises: underfitting vs. overfitting

Example is an instance of a general pattern: as models get more complex, training error keeps falling, but test error turns back up.

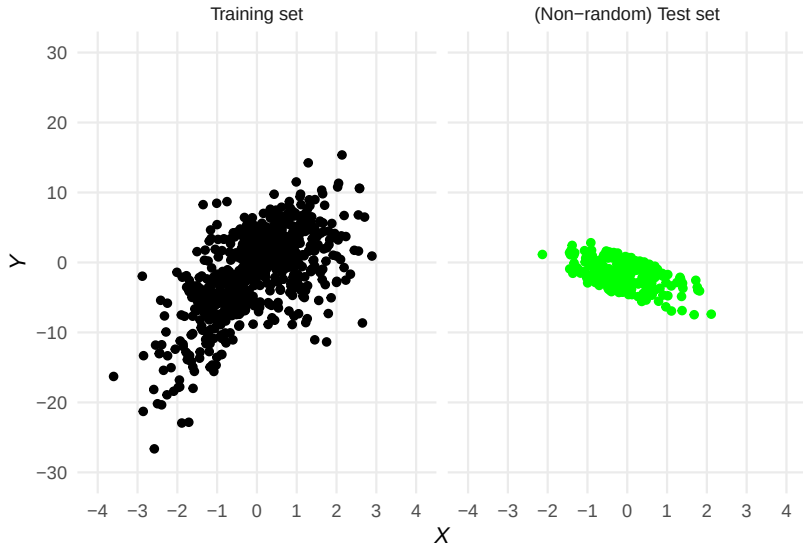


The general idea that there is a tradeoff between overfitting and underfitting is often called the **bias-variance tradeoff**.

Importance of random sampling



Importance of random sampling



Classification with trees

Decision trees

A very intuitive way to classify: decision trees.

Assume you have a dataset with $Y, X_1, \dots, X_J$.

Basic idea:

- For each predictor, find the split that best separates the values of Y (e.g., high X_1 versus low X_1).
- Split the data using whichever predictor's best split is best overall.
- Within each new subset, repeat the process.

This is (probably) how most humans think about classification!

Decision trees

Let's consider a simple example with a Y and X in a data set with six observations.

```
# A tibble: 6 x 2
```

	Y	X
	<dbl>	<dbl>
1	1	3.1
2	1	2.7
3	0	1.3
4	0	1
5	1	1.9
6	0	0.3

Decision trees

If you arrange by X , you can see a great split:

```
# A tibble: 6 x 2
  Y     X
<dbl> <dbl>
1     0  0.3
2     0  1
3     0  1.3
4     1  1.9
5     1  2.7
6     1  3.1
```

A decision tree is perfectly predictive. Here's the estimated model:

$$\hat{Y}_i = \begin{cases} 0 & \text{if } X_i < 1.6 \\ 1 & \text{if } X_i \geq 1.6 \end{cases}$$

Decision trees

Most datasets have much messier patterns in them.

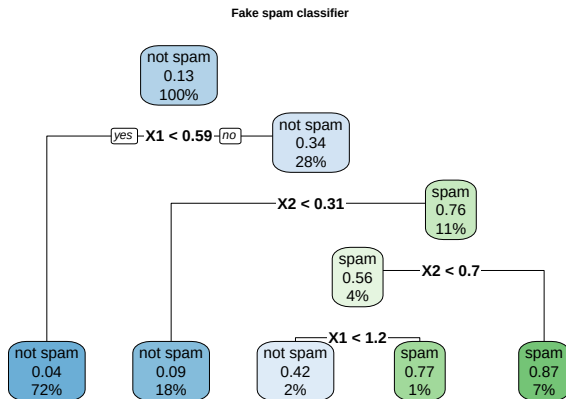
→ This will give much messier models.

A preview of the data for a messier example:

	Y	X1	X2
1	spam	0.9086071	1.10651071
2	not spam	2.3791234	-0.44262748
3	spam	1.2611259	0.06157537
4	spam	1.0928224	0.48917626
5	not spam	-0.5572497	-0.10269217
6	not spam	-0.3847158	2.03572050

Decision trees

```
library(rpart)
library(rpart.plot)
mod <- rpart(Y ~ X1 + X2, data = dft, method = "class")
rpart.plot(mod, main = "Fake spam classifier", cex = 1.5)
```



Anatomy of a decision tree

Some specialised terminology for trees:

→ **Nodes** are divided into two categories:

→ **Internal nodes**: places where splits/decisions occur.

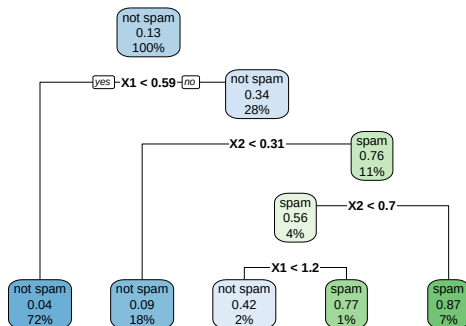
→ **Terminal nodes** or **leaves**: the ends of a tree that tell you the predicted classifications.

A tree is a diagram of the estimated model:

→ If you have some predictor values, you can find out the predicted class by navigating down the tree.

You can also write out the estimated model as a function.

Deriving a model from a decision tree



Note: as many rows as there are leaves. . .

$$\hat{Y}_i = \begin{cases} \text{not spam} & \text{if } X_{1i} < 0.59 \\ \text{not spam} & \text{if } X_{1i} > 0.59 \text{ and } X_{2i} < 0.31 \\ \text{not spam} & \text{if } 0.59 < X_{1i} < 1.2 \text{ and } 0.31 < X_{2i} < 0.70 \\ \text{spam} & \text{if } X_{1i} > 1.2 \text{ and } 0.31 < X_{2i} < 0.70 \\ \text{spam} & \text{if } X_{1i} > 0.59 \text{ and } X_{2i} > 0.70 \end{cases}$$

The splitting process

How do the splits happen in practice?

A **greedy, top down** approach with **binary splits**:

- For all predictors, find an *optimal binary split*.
- Whichever predictor's optimal binary split is best, that's the first split in the tree.

Then, with each subset created from this split, repeat the process.

Two issues:

1. how to measure “optimal” splits?
2. when to stop splitting (“growing the tree”)?

Measuring impurity

Split based on a measure of **impurity**:

- After a split, how mixed are response values within a new node?
- Goal: find splits that reduce impurity as much as possible.

For continuous response (regression trees)—use SSR.

For categorical response (classification trees)—use log-loss or Gini.

Regardless of measure of impurity, algorithm selects splits resulting in largest decrease in impurity.

In our first example: splitting at $X = 1.6$ leaves *no* impurity in either subset!

Growing and pruning decision trees

In principle, trees can keep growing (splitting) and get very deep.

Trees typically must stop growing if:

- All splits exhausted (every leaf is a single observation)
- All responses are identical in each leaf (no impurity left)

Trees can grow too big.

- Extreme example: when all splits are exhausted on large dataset.
- This model will be too complex, hard to use.
- It will also not perform well on new datasets (overfitting!!!)

Dealing with large trees

You could just stunt the growth of your tree, e.g.:

- Tell it to stop growing after certain # of splits.
- Tell it to stop growing when impurity is below a threshold.

Not ideal: what if important splits never get to happen?!

- Some important splits can happen deep in a tree.
- E.g., complex interactions may only emerge *after* earlier splits.
- Truncating early can miss these entirely.

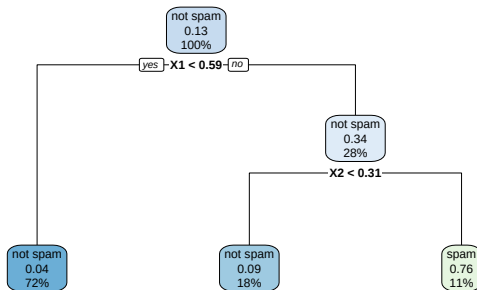
Better: grow the largest tree possible then **prune** it.

- Snip off least important splits.
- Choose tree that minimises: $\text{impurity} + \alpha \times (\# \text{ terminal nodes})$
- α is a hyperparameter (cp in R's `rpart`) – chosen by the analyst
- $\alpha = 0$ is no pruning

Dealing with large trees

```
pruned_tree <- prune(mod, cp = 0.1) # larger numbers prune more  
rpart.plot(pruned_tree, main = "Fake spam classifier (pruned)")
```

Fake spam classifier (pruned)



Pros and cons of trees

Decision trees are a very simple, intuitive way to classify.

→ Don't require any advanced statistics or DGP assumptions.

They also tend to capture complex, non-linear relationships and interactions between variables.

→ Common in social science!

Major downside of decision trees: models are *too complex*

1. Small changes in data can produce completely different tree.
2. Trees often fit the training data too closely.
3. Not robust to outliers or slight shifts in distribution.

In brief: they tend to overfit!

Building a robust model

Choosing your approach

Suppose you have a task in mind that's well suited for ML: you want to train a model to label some unlabeled data.

You have to choose some things:

- Which algorithm? Such as regression, decision tree, etc.
- What assumed DGP (e.g., for regression)?
- What **hyperparameters** do you want to set?

This process is called **model selection**. It's tempting to try many options and pick the best performer:

- Train 10 different algorithms on the training set.
- Evaluate performance of each on the test set.
- Choose whichever performs best in the test set.

This probably creates some (potentially serious) overfitting. Why?

Choosing your approach

You already know: the machine overfits by making choices based on how well its model performs in the training data.

By evaluating several algorithms on the test set, the human is now doing the same thing:

- She is making model selection decisions based on performance in the test set.
- And thus, tailoring the model to the test set.

Oops! This seems impossible: we want the best performer, but we can't identify it without risking overfitting.

Two things we can do:

1. Validation
2. Choose algorithms we *know* will do better without testing

Validation set

In addition to training and test sets, also create a validation set.

- The validation set is used to evaluate your model performance and choose a model.

A major advantage: it allows you to keep your test set untouched.

Seems like a small thing, but it is key:

The test set is only used one time to evaluate a model at the very end once the model has been chosen and estimated.

You cannot try to train another model after looking at test set performance.

At least now you will be able to see if you overfit!

Validation set

Doesn't solve all our problems, potentially creates new ones:

1. Same principles apply: can still overfit to the *validation set*.
2. What if you get an unusual validation set?
 - Remember the non-random test set from earlier: the same thing can happen *by chance*.
3. You're eating into your training set, reserving more of it for validation.
 - Machines aren't as good training with less data.
 - Less data also limits the kinds of models you can train.

Goal: train and validate on all (non-test) data

A core issue with this validation approach: wasting data.

- You didn't use all your data to train, nor did you validate your model on all your data.

If you validate on all your **non-test** data:

- You're guarding against drawing a "weird" validation set.

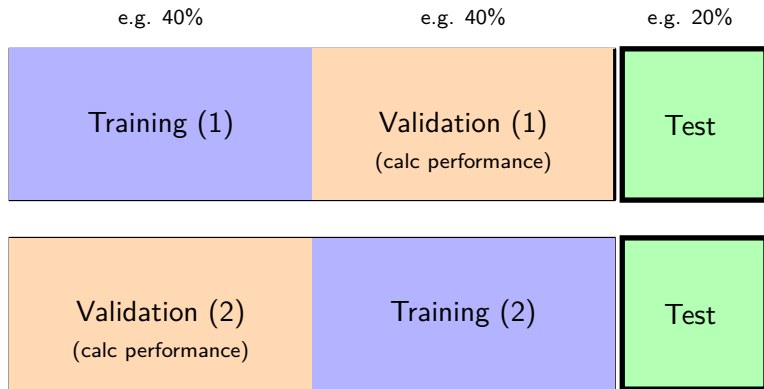
If you train with all your **non-test** data:

- You have a wider array of algorithms and models you can try.

Either way: the test set stays held out and stored away.

Cross-validation: the idea

Full (Labeled) Dataset



Cross-validation: the idea

Model selection with **cross-validation**:

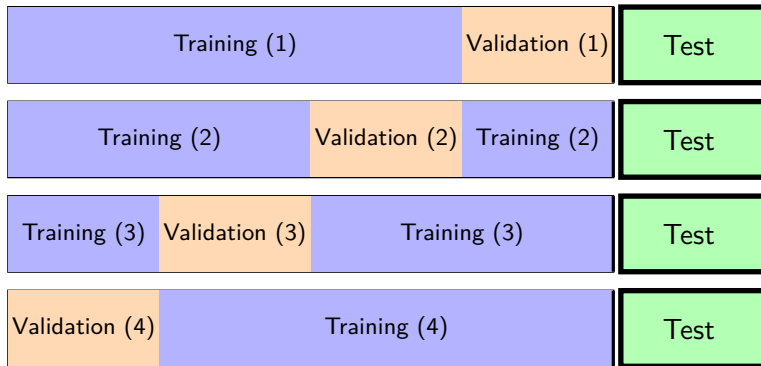
1. Split into training, validation and test
2. Pick an algorithm
3. Train on one half, validate on the other, then *switch*
4. Average performance metrics from the two validation sets
5. Repeat 2-4 for each algorithm you are considering
6. Choose the algorithm with the best average validation performance
7. Quantify performance on test set

Averaging matters: “unusually” good or bad performance in an unusual half gets pulled back toward the truth.

Now every observation is used to train *and* to validate. Why stop at one split?

Cross-validation: the idea

Full (Labeled) Dataset



Cross-validation

Notice: each round of training here uses a bigger dataset than each round in the two-fold split from before.

- This helps the machine out.
- It is easier for it to learn with more data.

I just showed you **four fold cross-validation** where you partition your data into 4 randomly created **folds**.

- For each fold k , you treat all the folds *except* k as a training set, fitting a model on it and evaluating performance on the held-out fold k .
- Repeat for each of the remaining folds.
- Average together the performance metrics across the folds.

More folds: (1) computation costs increase, but (2) each fold's training set is larger and more representative of full dataset.

Cross-validation

Some bad news. . .

You can still create overfitting if you use cross-validation, then tune your model and use cross-validation again (esp. with same folds).

The principle remains unchanged: you are at risk of overfitting when you tune your model in response to measured performance.

→ E.g., iteratively training on same training set in response to performance statistics from previous rounds of training.

Still: cross-validation is the best guard we have against overfitting during model selection.

Techniques for performance improvements

Tuning

Even after you commit to a general approach, you likely will need to **tune** your models.

Term is a bit ambiguous, but common agreement: it includes choosing **hyperparameters**.

- Hyperparameters control overall behaviour of your algorithms.
- E.g., instructions you supply about how to prune a tree (α)
- If you choose badly: underfitting or overfitting.

Common strategies (we won't explore in detail):

- **Grid search**: systematic, exhaustive
- **Random search**: faster, often good enough
- **Bayesian search**: learns where to look next

Typically: start broad and zoom in.

Cross-Validation to tune a hyperparameter

1. **Choose the hyperparameter**: e.g., α for decision trees
2. **Define a grid of candidate values**: such as $\alpha \in \{0.01, 0.1, 0.2, 0.5, 1\}$
3. **Set up k -fold cross-validation**: split training data into k folds
4. **Evaluate each candidate** value of the hyperparameter:
 - Train on $k - 1$ folds
 - Validate on the held-out fold
 - Repeat for all k folds
 - Average the performance across folds
5. **Select best hyperparameter**: choose value with best average validation performance
6. **Re-train the final model**: use the full training set and the selected hyperparameter

Choosing approaches with good properties

One way to guard against overfitting: only choose among approaches you already know to be pretty good.

- E.g., plot your data and choose DGPs that visually look right.
- In-sample performance is misleading for *final* evaluation, but useful for *initial* decisions.

Beyond judgement, two more principled tools improve performance:

1. **Regularisation**: make your model “more regular” by penalising excessive complexity.
2. **Ensemble methods**: blend multiple models to get the best of all of them.

Regularised regression

Recall that model complexity can create overfitting.

In regression: complexity can arise from too many predictors.

We can add a **penalty for model complexity** (e.g., too many predictors!), such that we now minimise:

$$\text{SSR} + \lambda \sum_{j=1}^J \beta_j^2 \quad \text{ridge regression}$$

or

$$\text{SSR} + \lambda \sum_{j=1}^J |\beta_j| \quad \text{lasso regression}$$

λ is the **penalty parameter** (a hyperparameter chosen with CV!)

→ The lasso and ridge penalties are often referred to as L1 and L2

Lasso vs. ridge regression

How does the penalty work?

- Algorithm minimises $SSR + \text{penalty}$, so large coefficients get pushed toward zero.
- The optimal way to minimise: push down coefficients that create large penalties.
- If $\lambda = 0$, both are equivalent to standard OLS.

Both types of regression penalise large regression coefficients.

- The idea: we're worried about some features causing our model predictions to swing wildly (too much variance).
- Coefficients that mostly fit noise get pushed toward zero; well-supported effects survive.

Lasso regression pushes some coefficients to zero; ridge does not.

- Lasso can be used for feature selection and to identify a regression when $J > n$.

Advantages and disadvantages of regularised regression

Advantages of regularised regression:

- Easy to interpret
- Works well when each feature helps predict Y on its own
- Works well when data is limited

Disadvantages:

- Features that only matter in combination (interactions, nonlinearities) must be added by hand (e.g., $I(X^2)$) – trees find these automatically
- Often outperformed by more complex methods

Ensembles (I): random forests

Key problem: want to make sensitive trees more robust.

Random forests improve on decision trees in two main ways:

1. **Bagging** (short for “bootstrap aggregating”)

- Train many decision trees on different bootstrap samples.
- Each tree sees a slightly different subset.
- Final prediction calculated across all trees. (average for regression, majority vote for classification)
- *Reduces variance through averaging.*

2. Random feature selection

- At each split in each tree, only a (small) random subset of predictors is used.
- Forces trees to explore different parts of the predictor space.
- *Reduces correlation between trees, improving power of averaging.*

Ensembles (I): random forests

Under the hood:

1. Each tree votes for a class label for each observation.
2. The forest counts votes, estimates the proportion of votes for each class.
3. The predicted class is the argmax of those proportions.

This is an ensemble method because the algorithm is combining results from a large number of trees.

- Creating a *forest* of trees
- Using “wisdom of the crowd” (or rather, the forest) to make better predictions.

What is the “model” of an estimated RF?

This is different than what we've seen before.

A few important distinctions:

1. Since this algorithm uses trees, there's no assumed DGP.
2. Since this averages over randomly generated trees, there's no overall tree to look at.
3. Since there is no overall tree, it's difficult (nearly impossible) to write out a model.
4. The algorithm initially produces predicted classifications; predicted probabilities are derived from the proportion of trees voting for each class.

Ensembles (II): stacking

Another common ensemble approach is called **stacking**.

One example, called **Super Learner**:

1. Train multiple base models (e.g., logistic regression, random forest, decision tree, etc.).
2. For each observation, collect *cross-validated* predicted probabilities from each base model.
3. Create a new dataset:
 - Predictors = model predictions
 - Response = true labels
4. Fit a “meta-model” (often a simple linear regression or logistic regression) using this new dataset.
5. The meta-model’s output is the final prediction.

Why cross-validated? If base models predict their own training data, the meta-model learns to trust whichever base model overfits the most.

Ensembles (II): stacking

Other ensembles (e.g., RF) use the same type of base model (decision trees)

- Stacking uses diverse model types – diversity is encouraged.
- Wider range of models allows for more opportunity to improve performance

The meta-model learns to trust some models more than others

- E.g., if the meta-model is a linear regression, $\hat{\beta}$ s are the weight the meta-model puts on each base model.
- Stacking is especially useful when base models are complementary such that they make different types of errors.